OOAD Lab 13

Name - Rithvik Rajesh Matta
Class - 6H
SRN - PES2UG23CS485

Question 1 — Adapter Pattern

Pattern: Adapter

Explanation: The Adapter pattern resolves interface mismatches by wrapping incompatible classes with a unified interface. Here, each lab class is wrapped by an adapter that implements `LabAdapter` (with `getPatientReport()`), so the hospital system talks to one consistent interface without touching the original lab classes.

Code:

```
// Existing external lab classes (cannot be modified)
class BloodLab {
    public String getBloodReport() {
        return "Blood Report from BloodLab";
    }
}

class PathLab {
    public String fetchLabResults() {
        return "Lab Results from PathLab";
    }
}

class DiagnosticCenter {
    public String retrieveDiagnostics() {
        return "Diagnostics from DiagnosticCenter";
    }
}

// Target interface expected by the hospital system
interface LabAdapter {
```

```
    String getPatientReport();
}

// Adapters for each lab
class BloodLabAdapter implements LabAdapter {
    private BloodLab bloodLab;
    BloodLabAdapter(BloodLab b) { this.bloodLab = b; }
    public String getPatientReport() { return bloodLab.getBloodReport(); }
}

class PathLabAdapter implements LabAdapter {
    private PathLab pathLab;
    PathLabAdapter(PathLab p) { this.pathLab = p; }
    public String getPatientReport() { return pathLab.fetchLabResults(); }
}

class DiagnosticAdapter implements LabAdapter {
    private DiagnosticCenter center;
    DiagnosticAdapter(DiagnosticCenter d) { this.center = d; }
    public String getPatientReport() { return
center.retrieveDiagnostics(); }
}

// Hospital system using the unified interface
public class HospitalSystem {
    public static void main(String[] args) {
        LabAdapter[] labs = {
            new BloodLabAdapter(new BloodLab()),
            new PathLabAdapter(new PathLab()),
            new DiagnosticAdapter(new DiagnosticCenter())
        };

        for (LabAdapter lab : labs) {
            System.out.println(lab.getPatientReport());
        }
    }
}
```

Output:

```
PS C:\Users\rithv\sem6\OOAD\lab 13> javac .\HospitalSystem.java
PS C:\Users\rithv\sem6\OOAD\lab 13> java HospitalSystem
Blood Report from BloodLab
Lab Results from PathLab
Diagnostics from DiagnosticCenter
PS C:\Users\rithv\sem6\OOAD\lab 13>
```

Question 2 — Proxy Pattern

Pattern: Proxy (specifically Virtual Proxy + Caching Proxy)

Explanation: The Proxy acts as an intermediary standing in front of the real `Video` object. It implements lazy initialization (loads only when needed) and caches already-loaded videos so repeated requests reuse the existing object instead of loading again — improving performance significantly.

Code:

```java
import java.util.HashMap;
import java.util.Map;

// Subject interface
interface Video {
    void loadVideo(String name);
    void playVideo();
}

// Real object — expensive to create
class RealVideo implements Video {
    private String name;

    public RealVideo(String name) {
        this.name = name;
        loadVideo(name); // Loading happens at construction
    }

    public void loadVideo(String name) {
        System.out.println("Loading video from disk: " + name);
```

```
    }

    public void playVideo() {
        System.out.println("Playing video: " + name);
    }
}

// Proxy — controls access and caches instances
class VideoProxy {
    private Map<String, RealVideo> cache = new HashMap<>();

    public void playVideo(String name) {
        if (!cache.containsKey(name)) {
            System.out.println("Cache miss — creating new RealVideo for: "
+ name);
            cache.put(name, new RealVideo(name));
        } else {
            System.out.println("Cache hit — reusing loaded video for: " +
name);
        }
        cache.get(name).playVideo();
    }
}

public class StreamingPlatform {
    public static void main(String[] args) {
        VideoProxy proxy = new VideoProxy();

        proxy.playVideo("Inception.mp4");
        System.out.println();
        proxy.playVideo("Interstellar.mp4");
        System.out.println();
        proxy.playVideo("Inception.mp4"); // Reused from cache
    }
}
```

Output:

```
PS C:\Users\rithv\sem6\OOAD\lab 13> javac .\StreamingPlatform.java
PS C:\Users\rithv\sem6\OOAD\lab 13> java StreamingPlatform
Cache miss ? creating new RealVideo for: Inception.mp4
Loading video from disk: Inception.mp4
Playing video: Inception.mp4

Cache miss ? creating new RealVideo for: Interstellar.mp4
Loading video from disk: Interstellar.mp4
Playing video: Interstellar.mp4

Cache hit ? reusing loaded video for: Inception.mp4
Playing video: Inception.mp4
PS C:\Users\rithv\sem6\OOAD\lab 13>
```

Question 3 — Flyweight Pattern

Pattern: Flyweight

Explanation: The Flyweight pattern separates **intrinsic state** (shared, immutable: texture, color, type) from **extrinsic state** (unique per instance: x, y position). Thousands of trees share a single `TreeType` flyweight object instead of each storing their own copy of texture/color/type, drastically reducing memory usage.

Code:

```java
import java.util.HashMap;
import java.util.Map;
import java.util.ArrayList;
import java.util.List;

// Flyweight — stores shared intrinsic state
class TreeType {
    private String type;
    private String color;
    private String texture;

    TreeType(String type, String color, String texture) {
        this.type = type;
        this.color = color;
        this.texture = texture;
```

```
    }

    public void draw(int x, int y) {
        System.out.println("Drawing " + type + " [color=" + color +
                           ", texture=" + texture + "] at (" + x + ", " +
y + ")");
    }
}

// Flyweight Factory — ensures shared instances
class TreeFactory {
    private static Map<String, TreeType> treeTypes = new HashMap<>();

    public static TreeType getTreeType(String type, String color, String
texture) {
        String key = type + color + texture;
        if (!treeTypes.containsKey(key)) {
            System.out.println("Creating new TreeType: " + type);
            treeTypes.put(key, new TreeType(type, color, texture));
        }
        return treeTypes.get(key);
    }

    public static int getTotalTypes() { return treeTypes.size(); }
}

// Context — stores extrinsic state (position) + reference to flyweight
class Tree {
    private int x, y;
    private TreeType treeType;

    Tree(int x, int y, TreeType treeType) {
        this.x = x;
        this.y = y;
        this.treeType = treeType;
    }

    public void draw() {
        treeType.draw(x, y);
    }
```

```
}

public class GameEnvironment {
    public static void main(String[] args) {
        List<Tree> forest = new ArrayList<>();

        // Thousands of trees — but only a few shared TreeType objects
        TreeType oak   = TreeFactory.getTreeType("Oak",   "Green",
"RoughBark");
        TreeType pine  = TreeFactory.getTreeType("Pine",  "DarkGreen",
"SmoothBark");
        TreeType maple = TreeFactory.getTreeType("Maple", "Orange",
"FlatLeaf");

        forest.add(new Tree(10, 20, oak));
        forest.add(new Tree(30, 40, pine));
        forest.add(new Tree(50, 60, oak));   // reuses oak flyweight
        forest.add(new Tree(70, 80, maple));
        forest.add(new Tree(90, 10, pine));  // reuses pine flyweight

        for (Tree t : forest) t.draw();

        System.out.println("\nTotal trees: " + forest.size());
        System.out.println("Total TreeType objects in memory: " +
TreeFactory.getTotalTypes());
    }
}
```

Output:

```
PS C:\Users\rithv\sem6\OOAD\lab 13> javac .\GameEnvironment.java
PS C:\Users\rithv\sem6\OOAD\lab 13> java GameEnvironment
Creating new TreeType: Oak
Creating new TreeType: Pine
Creating new TreeType: Maple
Drawing Oak [color=Green, texture=RoughBark] at (10, 20)
Drawing Pine [color=DarkGreen, texture=SmoothBark] at (30, 40)
Drawing Oak [color=Green, texture=RoughBark] at (50, 60)
Drawing Maple [color=Orange, texture=FlatLeaf] at (70, 80)
Drawing Pine [color=DarkGreen, texture=SmoothBark] at (90, 10)

Total trees: 5
Total TreeType objects in memory: 3
PS C:\Users\rithv\sem6\OOAD\lab 13>
```

Question 4 — Facade Pattern

Pattern: Facade

Explanation: The Facade pattern provides a single simplified interface (`completePurchase()`) that internally coordinates all the complex subsystem calls — `login()`, `selectProduct()`, `makePayment()`, and `trackOrder()`. The client no longer needs to know the order or details of these steps; the Facade handles everything, reducing coupling and complexity.

Code:

```java
// Subsystem classes
class AuthService {
    public void login(String user) {
        System.out.println("User logged in: " + user);
    }
}

class ProductService {
    public void selectProduct(String product) {
        System.out.println("Product selected: " + product);
    }
}
```

```java
class PaymentService {
    public void makePayment(double amount) {
        System.out.println("Payment of Rs." + amount + " processed
successfully.");
    }
}

class OrderService {
    public void trackOrder(String orderId) {
        System.out.println("Order " + orderId + " is now being tracked.");
    }
}

// Facade — single unified interface for the complex purchase flow
class ECommerceFacade {
    private AuthService auth       = new AuthService();
    private ProductService product = new ProductService();
    private PaymentService payment = new PaymentService();
    private OrderService order     = new OrderService();

    public void completePurchase(String user, String productName,
                                 double amount, String orderId) {
        System.out.println("=== Starting Purchase Flow ===");
        auth.login(user);
        product.selectProduct(productName);
        payment.makePayment(amount);
        order.trackOrder(orderId);
        System.out.println("=== Purchase Completed Successfully ===");
    }
}

// Client — interacts only with the Facade
public class ECommerceApp {
    public static void main(String[] args) {
        ECommerceFacade facade = new ECommerceFacade();
        facade.completePurchase("Alice", "Wireless Headphones", 2999.99,
"ORD-00421");
    }
}
```

Output:

```
PS C:\Users\rithv\sem6\OOAD\lab 13> javac .\ECommerceApp.java
PS C:\Users\rithv\sem6\OOAD\lab 13> java ECommerceApp
=== Starting Purchase Flow ===
User logged in: Alice
Product selected: Wireless Headphones
Payment of Rs.2999.99 processed successfully.
Order ORD-00421 is now being tracked.
=== Purchase Completed Successfully ===
PS C:\Users\rithv\sem6\OOAD\lab 13>
```